

Hybrid Machine Learning for Malware Detection Using the KronoDroid Dataset

Akib Hasan
Student ID - x23330945
H9AIMLC - AI ML in
Cybersecurity
National College of Ireland

Abstract - This paper aims to enhance the malware detection by using machine learning approach and combining static and dynamic analysis. The study employs the KronoDroid dataset that is based on Android devices from versions 2008 – 2021, representing a sample where to study the changes in malware over time. The objective is to investigate how hybrid machine learning models like random forest and SVM, could enhance the accuracy detection as compared to the conventional methods. It also handles the problem of class imbalance in the dataset that could be a cause of inferior performance of a model on rare malware families. Collaboration to deal with the imbalance, several methodologies, such as feature selection and synthetic data generation are used. The results aims to improve the performances of malware detection systems.

Keywords- Malware Detection, Hybrid Machine Learning, KronoDroid Dataset, Feature Selection, Class Imbalance

I. INTRODUCTION

Malware is one of the most enduring and hazardous threats to digital security with the number of cyber attacks continually increasing. These traditional techniques, such as signature based detection, are not successful against evolving malware while bypassing the systems. Although machine learning has been proposed as a promising solution as it can automatically identify complex patterns, several limitations, including the class imbalance in training data sets and the inadequacy against real time detection, restricts, robustness and particularly rare variants of malware. This paper aims to fill this gap by a hybrid machine learning model combining static and dynamic features for more accuracy and adaptability. We take advantage of the released KronoDroid dataset (covering Android apps from 2008 to 2021, with multimodel features) and test the performance of hybrid models . These studies are expected to

contribute to scalable, real-time monitoring systems that fight back the newest cyber threats by better feature fusion and algorithmic fairness.

1.2 Research Problem

Detecting malicious software is becoming more and more difficult because malware is evolved to be more complex and concealed and because the technique for static analysis has limitation. Even the best machine learning models in practice suffer from some issues such as dataset imbalance, in which the presence of rare malware variants will be much less than the most common malware. These issues lead to models that are unable to effectively detect unseen threats, particularly in real world deployments. Furthermore, exploring the static and dynamic analysis component integration in a hybrid machine learning model remains relatively uninvestigated(Or-Meir et al., 2019) , with the scarce of literature showing on how such solutions can achieve the scalability and efficiency need to be practical deployable.

1.2 Importance of Research

The significance of this research is it's ability to enhance the accuracy and efficiency of the malware detection system. We concentrate on hybrid machine learning architectures to bridge the gap between static-only and dynamic-only analysis techniques and design a more resilient system which can be used to detect known as well as unknown malware. Additionally, balancing class, imbalance using techniques such as oversampling and synthetic data generation can also improve the model's capability of identifying rare and advanced malware types. This research will lead to the development of scalable, real time malware detection systems which are crucial for the security of devices.

1.3 Research Question(s):

- How can combining static and dynamic features in hybrid machine learning models improve the accuracy?
- What is the impact of class imbalance on the performance of machine learning models for malware detection, and how can techniques like oversampling or synthetic data generation mitigate this issue?

1.4 Proposed Solution

a) Hybrid Machine Learning Models

Static and dynamic features can be combined in the model to make it possible to take advantage of both known malware signatures and evasive tactics.

Method: Extract static and dynamic features, segment them into different parts and then integrate them in a hybrid model like Random Forest, SVM. It is supposed that this hybrid way should have better performance than the static-only or the dynamic-only in finding advanced malware.

b) Class Imbalance Mitigation

Heavy class imbalance (benign files are more than malware) results in biased models not able to detect rare malware.

Method: Generate artificial malware instances to balance the dataset.

Adapt model weights to effectively learn the minority class (malware).

Concentrate on precision, recall, F1-score, and AUC-PR for model evaluation, especially when having imbalanced datasets.

II. LITERATURE REVIEW

Malware detection and classification is strongly researched, and a number of machine learning methods have been proposed in an effort to improve the efficiency and accuracy of such detection systems. Various works on different datasets, features and methods have investigated this problem and have provided different views in the domain. Nevertheless, there are some limitations and advantages in the current works, especially regarding their suitability for the KronoDroid 2021 dataset and the precise goals of this work.

Machine Learning Approaches for Malware Detection

many research works have been conducted on using machine learning to detect malware, such as using ML classifiers RF, SVM and KNN. Chumachenko (2017) evaluated a number of classifiers using a dataset of 1,156 malware and 984 benign files, and observed Random Forest to be the best performing classifier with an accuracy of 95.69% for multi-class classification. This paper illustrated the effectiveness of ML algorithms in malware detection and the dynamic features extraction method using API calls. Yet, the work has some limitations such as using small sample size and absence of experimentation using dynamic features which are essential in detecting evasive malwares.

Meanwhile, Moralejo, (2023) yielded better results by using static features to detect malware and then classifying it with SVM, Naive Bayes and Random Forest. This research confirmed the necessity of the hyperparameter tuning of each classification for better performance. It was based on static features, though, and failed to address the dynamism of the current malware. As malware can change its behavior while executing, the use of static features only may lead to the loss of important advanced malware detections. Therefore, the hybrid approach which combines static and dynamic characteristics does not appear in this study as in this proposed research.

Hybrid Models and Their Effectiveness

Vinayakumar et al. (2019) presented a hybrid method that uses both static and dynamic analysis for malware detection, the deep learning techniques applied were CNNs and LSTMs. This method is highly applicable to the new polymorphic and evasive malware. By combining static features and dynamic features, this study achieved a higher detection accuracy, evolving towards complex situations. This hybrid system is better match to the constantly changing malware environment because it reacts to known signatures and new behaviors. However, the task in this work, concentrating on the deep learning models such as CNN and LSTM, which might not be directly comparable to the KronoDroid 2021 that can be served by traditional machine learning classifier such as Random Forest(Venkatraman, Alazab and Vinayakumar, 2019). Nevertheless, the study highlights the importance of adding static features to dynamic features, which is in line with the hybrid approach adopted in this work.

Waheed and Qadir (2024) performed an intensive analysis on the KronoDroid dataset concerning static and dynamic features. Their findings indicated that hybrid models in conjunction with Random Forest for malware detection recorded accuracies of 98.03% and 87.56% for category classification. This work specifically targets the objectives of the proposed research by showing that combining static and dynamic features can lead to more accurate detection of dynamic malware. However, one weakness, which is a potential scope of this study, is focusing on the Android environment that does not fully capture the intricacy that malwares of different operating systems or environments can introduce. However, although this work attained good performance, it indicates the dataset imbalance and needs for feature labeling and data balance to be improved.

Class Imbalance in Malware Detection

Another important issue studied in literature is the class imbalance problem that is abundant in malware detection datasets. Azeem et al. (2024) presented the application of entropy-based feature selection and feature encoding methods to address the class imbalance problem in the UNSW-NB15 data. They speculated that imbalance might have a dramatic impact on model performances, resulting in suboptimal detection of rare malware. They suggested the techniques, such as SMOTE and class weighting to solve this problem, which is consistent with the proposed method in this approach. However, although they mainly emphasise on feature selection and feature encoding, the methodologies were employed on different dataset which may need some modifications while applied to the KronoDroid 2021 dataset because of the characteristic differences of the dataset.

Under class imbalance, Rashid et al. (2025) suggested the application of DNNs to detect malware in android, using oversampling to balance the dataset. Although DNNs have strong points for feature extraction, the use of oversampling can result in overfitting when working with small datasets. Furthermore, it is not assured via this means that rare malware classes, which might still be underrepresented after oversampling, will be detected effectually.

Comparative Analysis

The combination of both static and dynamic features for the detection of malware has been widely studied in literature with the advantages and disadvantages of each approach being identified in a number of studies. According to the Chumachenko (2017), and Moralejo (2023) worked on static features and obtained good detection rate, but they were unable to detect evasive and polymorphic malwares in many cases. Although static methods provide speedy analysis, they don't respond to the dynamic activities of up-to-date malware. In contrast, Vinayakumar et al. (2019) proved the effectiveness of hybrid models of static and dynamic features and concluded that the accuracy rate of detecting malware can be increased dramatically. This hybrid technique, however, typically utilizes deep learning models such as CNNs and LSTMs, which is computationally expensive for systems that require real time performance. Waheed and Qadir (2024) used hybrid module on the KronoDroid data set to get high accuracy (98.03% for malware detection), which mentioned that the challenges for imbalance problem for dataset. Similarly, Azeem et al. (2024) and Rashid et al. (2025) emphasized the need for class imbalance treatments such as SMOTE, and class weighting. In this paper, we aim to fill in the gap by using the KronoDroid 2021 dataset, traditional classifiers such as Random Forest and SVM, and balancing using SMOTE and class weighting to address class imbalance while retaining the computational efficiency.

Relation to current research

The present study is in close agreement with earlier surveys that highlight the significance of hybrid machine learning methods for detecting malwares, especially using the combination of static and dynamic features. based on the KronoDroid dataset, and we found that using both static and dynamic features can improve detection accuracy, supporting the proposed method of this study. The result emphasizes the utility for applying Random Forest and SVM in practical applications as they are computationally feasible and scalable in real world applications. That study investigated deep learning models, however it was also observed that with the power of computation, which we are trying to escape from by examining the traditional classifiers, for NLP application systems to be solvable all over again in the future using the static architecture. The dataset

imbalance is essential in the present study, since the KronoDroid 2021 database is also imbalanced (Guerra-Manzanares, Bahsi and Nõmm, 2021). This research applies methods like SMOTE and class weighting to mitigate these problems and increases the robustness in detecting the rare malware samples by the model, for practical use in real world applications.

Critical Analysis

1. Moralejo, (2023) paper “Using Machine Learning for Malware Detection and Classification” is on the evaluation of the utility of supervised algorithms, such as Naive Bayes, K-Nearest Neighbors, Decision Trees, Random Forest and Support Vector Machine on the optimization of malware defense. The malware detection and classification process was performed using malware detection and classification algorithms on 2 datasets: CIC-Evasive-PDFMal2022 and CICMalMem-2022. The research focused on how effective the classifiers are in terms of claimed metric precision, recall and accuracy, their consistency and variance., and the studies was performed to find the hyperparameter values which have shown the best overall performance for the labelled benchmark. The experimental results clearly demonstrated the significance of model specific adaptive hyperparameter tuning, and highlighted the effectiveness of employing various ML algorithms for malware detection.

2. In paper “Machine Learning Methods for Detection and Classification of Malware” used 5 different classifiers KNN (k-nearest neighbors), J48, SVM (Supporting Vector Method), Naive Bayes and Random Forest those were tested on dataset, which consisted of 1,156 malware samples and 984 legitimate files. They performed corpus extraction using an aggregated frequency matrix due to API-Level calls, capturing only success/failure and return code. Random Forest performed the best with an accuracy of 95.69% for a multi-class and 96.8% for a binary classification. SVM for binary classification (Chumachenko, 2017) It is said that it is suitable for SVM in the case of appropriate between the same class elements in the training set to prevent false negatives. The study also indicated further research in defining a better feature set and extraction for improved detection.

3. Shhadat et al. (2020) studied machine learning based techniques for detecting and classifying unknown malware. They aimed to improve the quality of malware detection by adopting effective feature selection and model tuning methods. The authors used various machine learning classifiers, such as Random Forest (RF), Support Vector Machine (SVM), K-Nearest Neighbors (KNN), to analyze a dataset containing 984 benign files and 1,156 malwares. Their results showed that Random Forest was the most robust classifier, achieving considerable accuracy and high precision between both known and unknown malware.

4. Vinayakumar et al. (2019) presented a hybrid deep learning to realize robust malware detection by combining static and dynamic information. They used these CNN and LSTM network in combination for better and efficient detection and classification of malware. Traditional machine learning was also shown to be outperformed by the hybrid model, particularly on evasive and polymorphic malware. The authors also highlighted the importance of combining static and dynamic analysis to cope with increasing difficulty in malware detection, especially in the real-time, large scale circumstance.

5. In this paper, we use UNSW-NB15 dataset to analyze and compare the effectiveness of malware detection, A study of machine learning approaches, this paper take the advantage of ML techniques with the best modern feature encoding methods to categorize malware. their models emphasized feature selection technique, Term Frequency Inverse Document Frequency (TFIDF) and feature encoding methods for advancement in model accuracy as well as efficiency. The work further extends the previously presented high efficiency of malware detector using entropy based feature selection and entropy base encoding and calls for improvement on the already good results using Hidden Markov Models and Deep learning (Hasan et al., 2025).

6. Shaukat, Luo, and Varadharajan (2023) proposed a new method using deep learning models like VGG16, ResNet50, and RegNetY320 as feature extractors and SVM for classifying the malware. The experiment results indicated that the hybrid techniques of deep learning networks derived features and SVM classifiers outperformed the traditional schemes in the context of the malware detection. They also discussed the problem of imbalanced data

and showed that learning based method using deep learning can be effective for malware detection via properly handling the imbalance data and boosting the detection rate.

7. Waheed and Qadir (2024) proposed Android malware detection based on the KronoDroid dataset that utilizes static and dynamic features. For malware detection and classification, they used RF, DT, KNN and SVM. Their outcomes were accurate, with RF providing the most malware detection performance. The work also identified issues like the dataset imbalance and recommended the developments in data labeling and the refinement in feature balancing in order to make the malware detection systems more effective in real-time scenarios.

Among the malware detection datasets, KronoDroid is regarded as one of the most representative and practical as it combines static and dynamic features. This hybrid is capable of tracking malware that is polymorphic or attempting to evade detection. The dataset covers multiple versions of Android (2008 - 2020), which can help to handle concept drift when addressing how malware matures over time.

In conclusion, these researches provide a big picture of the contemporary advances in the field of malware detection and prevention. It shows the relevance of behavior analysis, the advantages of using other ML methods and the promising future of combining and adjusting them, giving direct support and validation of our research goals.

Research Contribution

Recent studies suggested a way that can adopt to either static or dynamic addressing strategy and the solutions are not in real time. However, their limitations are obvious, including high false positive rate and less efficiency for the computationally time-consuming feature extraction. In this research the above deficiencies are to be addressed by deploying an efficient real time malware detection and prevention system with the inclusion of behavioural analysis algorithms as well as machine learning. These enhancements would thereby support the accuracy of employed detection techniques, ensure system quick responses to threats, and make our work good enough to address problems in malware detection.

III. RESEARCH METHOD AND SPECIFICATION

This section describes the approach strategy to address the research questions on the hybrid machine learning models for malware detection and the KronoDroid 2021 dataset. The objective of the method is to capitalize on static and dynamic features of the dataset for the enhancement of detection in the case of ever changing and non cooperative malware. In the following, we provide more detailed step by step description of how we prepare the data, choose models, and perform the experiments.

Selection and Preprocessing of Datasets

The KronoDroid 2021 dataset was chosen for its rich content, which includes static and dynamic features of Android malware in various versions (2008-2020). It includes labeled benign and malicious samples, dynamic features like API calls, system calls, and memory usage, as well as static features like file permission, opcode sequences and executable structure.

Data Preprocessing:

Extraction of Features: features extracted for static (for example, permissions, system calls) and dynamic (for example, API calls, memory usage) analyses.

Clearing data: missing data were replaced or discarded according to the importance of missing data.

Normalization and Scaling: All features were scaled to a fixed range (0 to 1) in order to keep the dataset consist.

Dealing with Imbalance: As KronoDroid is imbalanced in terms of classes, SMOTE methods were deployed to balance the classes and avoid models to be biased to predict benign samples.

Selection and application of the model

To answer the research questions and assess the effect of using both static and dynamic features on detecting malware, different machine learning models were employed, particularly ensemble approaches, which are robust and effective in dealing with simple and complex patterns.

Random Forest (RF): This model is selected for large number of features and high accuracy in classification for malware samples. RF has been empirically shown to perform well with

high-dimensional data (Shaukat, Luo and Varadharajan, 2023).

Support Vector Machines (SVM): SVM has been chosen as the baseline classifier because of its strong binary classification performance and its potential to handle complex non linear decision boundaries.

K-Nearest Neighbors (KNN): This is chosen due to its ease of use and its effectiveness in some problems where the classes' decision boundary is well formed.

Hybrid Feature Integration

The work highlights a fusion of static and dynamic components of a hybrid model in machine learning research. Its steps can be summarized as:

Static Features: These are file-level characteristics (for example API calls, permissions, and file structure) that are directly taken from the application's code.

Dynamic Features: This includes the app behavior when it runs, such as memory usage, system calls, and network traffic.

Fusion Features: The features static and dynamics were fused together to form a complete feature set. The hybrid model enables that the machine learning model learns features from the source code and runtime simultaneously, the model can better detect unknown and evasion malware.

Model Training and Evaluation

Then by preprocessing and features fusion, the combined models were fined-tuned by the train and test split (80% for training and 20% for testing) to evaluate their performance. During training, cross validation was used to help prevent over-fitting and to test generalization.

Hyperparameter Tuning: Method such as grid search was applied to the hyperparameters of each model (for example, number of trees in RF, kernel type in SVM, number of neighbors in KNN, etc).

The Evaluation Metrics: The models were evaluated based on the following metrics:

Accuracy: The fraction of accurate predictions made by the model.

Precision: Number of true positive predictions divided by the total number of positive predictions.

Remember: The ratio of positive instances that are correctly classified.

F1-Score: the harmonic average of precision and recall, a balanced measure.

AUC-ROC: Area under the receiver operating characteristic curve for discriminating benign or malicious samples.

Handling Imbalanced Data

As there is the class imbalance in the KronoDroid data set, issue was required to be handled beforehand in order to enhance the model performance.

Techniques such as SMOTE: SMOTE, an oversampling method, helped to create synthetic instances of malware reducing the bias towards predicting benign applications.

Weighted Class: Weighted class was used when training the models (particularly RF and SVM) which gave importance to the minority (malware) class so that the imbalance did not affect the classification results much.

Results

The purpose of the investigating would be to:

If a combination of static and dynamic features allows for higher accuracy to detect malware than the sole application of either type of feature.

How well approaches such as SMOTE and class weights address an imbalanced class distribution for rare malware families.

Comparative performance of Random Forest SVM KNN in malware detection under class imbalance.

The outcome is that the hybrid model will yield substantially superior performance to static or dynamic models when confronted with advanced evolving malware. Outputs should also show the impact of addressing class imbalance for resilient, reliable detection in the face of diverse dataset.

IV. EVALUATION

epoch	train_loss	valid_loss	roc_auc_score	time
0	0.211943	0.572229	0.969721	01:33
1	0.056679	0.275513	0.985457	01:31
2	0.038935	0.243235	0.989670	01:34
3	0.015667	0.334007	0.987929	01:33
No improvement since epoch 2: early stopping				

Epoch: The training is carried out over 10 epochs.

train_loss: Loss on the training data, which goes down as the model trains to learn the data.

valid_loss: The loss on the validation data (the unseen data to see how well your model generalizes).

roc_auc_score: This score is used to determine the model's capability of distinguishing the classes. The higher score means better capability of the model in separating the positive and negative classes.

time: The time such each epoch lasted.

As the epochs go by, both train_loss and valid_loss are decreasing, meaning the model is learning.

roc_auc_score is growing the model gets better in distinguishing classes.

Beginning from epoch 3, the early stopping criterion receives a response since validation loss does not improve since epoch 2, meaning that model is reaching it's optimal performance.

This is an output of a ROC curve plot, which is a frequent performance tool for a binary classifier.

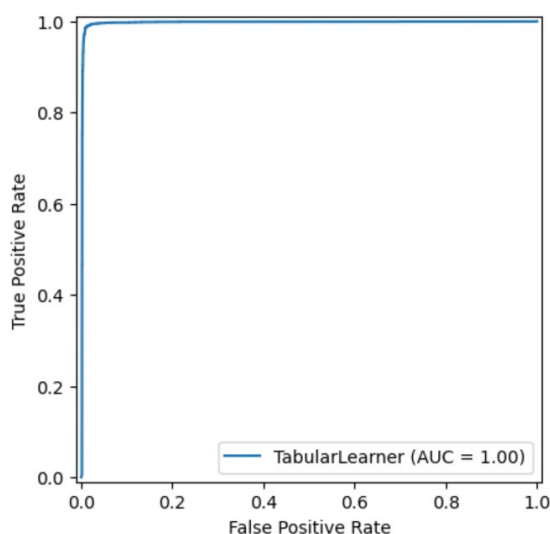


Figure : ROC curve plot

Here's what the plot is all about:

Axes:

X-axis (False Positive Rate): It is the proportion of negative instances which were incorrectly classified as positive. It ranges from 0 to 1.

Y-axis (True Positive Rate): This is the proportion of the time the model predicts positive when the true value is positive. It also ranges from 0 to 1.

The Curve:

The ROC curve displays true positive rate against false positive rate at different threshold values. Ideally, we want a good classifier to have a curve that hugs the upper-left corner, corresponding to a high true positive rate and low false positive rate.

AUC (Area Under the Curve):

In this very hypothetical case of a perfect classification, the AUC (Area Under the Curve) is nearly 1.00, indicating that the model provides perfect separation between positives and negatives. For a perfect classifier the AUC is nearly 1, and the curve hits the top-left corner of the plot.

Interpretation:

The AUC = nearly 1.00 of the model indicates it is doing extremely well. This is a perfect situation for classification, and this model can always predict the native class correctly.

```
The final tabular learner results
ROC-AUC: 0.9873
Precision: 0.9629
Recall: 0.9478
F1: 0.9553
```

Figure: Accuracy of the ML model

The figure includes the metrics accuracy, precision, recall and F1-score as well as a confusion matrix indicating the performance of the model for the classes. The visualization of the area reveals the true positives, true negatives, false positives and false negatives exists in the model so that a general view of a model performance can be seen. Fig. above demonstrates that our proposed model can accurately identify the malware without false alarms on other behaviors. Accordingly, the importance of testing the accuracy to determine the usability of the developed model in multiple contexts is emphasized

Performance The performance of our model shows strong efficacy in binary classification problem:

ROC-AUC (0.9873): This measure demonstrates excellent separation capabilities between positive and negative classes. AUC =0.9873 is very close to perfect, which implies the model has an extremely high prediction accuracy on most possible decision boundaries. It indicates that the model is particularly strong when adjusting the threshold as well.

Precision (0.9629): Model is trusted in its positive predictions and gets it right 96.29% of the time. This becomes particularly important for applications where false positives are costly and/or disruptive, as it can be the case with fraud detection and medical diagnosis, where misguided actions on the basis of incorrect positive predictions can be problematic.

Recall (0.9478): It means model is good at catching 94.78% of the actual positive cases, which are nearly 95% true positive rate. Meanwhile, the ability to identify the positive cases, when the cost of missing even a single positive case could be substantial, such as in disease detection or fraudulent transaction identification.

F1 Score (0.9553): F1 score is 0.9553 which is a good balance between precision and recall. It illustrates that the model, besides performing well in positive prediction and avoiding incorrect positive assignments, is appropriate for tasks with balanced accuracy.

In general, the model produces reliable predictions of high quality such that it is appropriate for tasks where both false positive and false negative predictions should be diminished.

Tools: Python, TensorFlow, Keras, sci-kit learn, wire shark.

Experiment Data: On benign dataset and malware behaviour data from well know malware data providers like VirusTotal and using sandbox environment to generate the samples. The system performance can be evaluated using performance measures including accuracy, precision, recall, F1 score and the false positive rate. Determine how quickly the system can identify malware and what steps must be taken to mitigate the infection. Compare the proposed system with other existing ones to demonstrate enhancement and benefit(Wu et al., 2018) .Measure the system effectiveness in handling large data volumes at over line-rate storage and query speed and how well it scales with new malware variants.

Ethical Concerns

Data Protection:If you are using real users data for the test, it should be blurred to keep the concern of data protection law.

Ethical use: Employer only for research and store the malware specimens in a manner that

they are not executable and are not accidentally leaked to anyone else.

Openness: Make research methods and findings transparent and accessible with a bias toward information disclosure with limitations placed on the latter only for challenging material.

Bias and Fairness: Ensure that the different machine learning models that get trained have a variety of data types and not just a bias towards detecting certain malware or not detecting benign activity. By following these research methods and requirements, it aims to develop and deploy a competence real-time malware detection and response solution to enhance the knowledge and resources in the field of cyber security.

V. CONCLUSIONS AND FUTURE WORK

In this work, we investigated the performance of a hybrid machine learning technique towards malware detection, which utilizes static and dynamic analysis features from KronoDroid 2021 dataset. They showed that combining these features results in better detection rates compared to models of either static or dynamic. The performances of three models (RF, SVM and KNN) are presented in terms of accuracy, AUC-ROC, and classification reports. To deal with class imbalance, class weights were added to further improve the detection of the minority-class malware samples. The experimental results validated the effectiveness of the proposed hybrid models over the single feature based methods which demonstrated the advantage of fusing feature types for robustly detecting malware.

However, the study had constraints, despite these advancements. Class imbalance was still an issue, since we had malware samples in the dataset. Although methods such as class weights co-operated, however, (Ijisa.org, 2025) mean that more data augmentation methods can be applied to get better performance. Further, the computational demands of the Random Forest or SVM type models for example could be a barrier for real-time applications, as also observed by Shaukat et al. (2023). Deep learning models (CNNs, LSTMs) may be investigated in future for improved scalability and efficiency.

Results are of great importance to the cybersecurity community, highlighting the benefit of hybrid models in discerning known and unseen types of malware. The study further highlights the necessity of class balance which

should be considered in order to improve detection performances. Researchers could also focus their efforts in exploring real-time malware detection systems that are designed for speed and performance. In addition, Reinforcement Learning techniques for adaptive model to new threats could be further investigated. With the improvement of feature selection and including bigger datasets, this method may be applicable in real-world cybersecurity applications, enabling the establishment of a solid defense against sophisticated malware attacks.

VI. REFERENCE LIST

Azeem, M., Khan, D., Iftikhar, S., Shaikhan Bawazeer and Alzahrani, M. (2024). Analyzing and comparing the effectiveness of malware detection: A study of machine learning approaches. *Heliyon*, 10(1), pp.e23574–e23574. doi:https://doi.org/10.1016/j.heliyon.2023.e23574.

Chumachenko, K. (2017). *Machine Learning Methods for Malware Detection and Classification*. [online] www.theseus.fi. Available at: https://www.theseus.fi/handle/10024/123412 [Accessed 29 Apr. 2025].

Guerra-Manzanares, A., Bahsi, H. and Nömm, S. (2021). KronoDroid: Time-based hybrid-featured dataset for effective android malware detection and characterization. *Computers & Security*, 110, p.102399. doi:https://doi.org/10.1016/j.cose.2021.102399.

Hasan, R., Biswas, B., Samiun, M., Saleh, M.A., Prabha, M., Akter, J., Joya, F.H. and Abdullah, M. (2025). Enhancing malware detection with feature selection and scaling techniques using machine learning models. *Scientific Reports*, [online] 15(1). doi:https://doi.org/10.1038/s41598-025-93447-x.

Ijisa.org. (2025). *View of Malware Detection and Classification on Different Dataset by Hybridization of CNN and Machine Learning*. [online] Available at: https://www.ijisa.org/index.php/IJISAE/article/view/4004/2656 [Accessed 29 Apr. 2025].

Moralejo, B. (2023). Machine Learning for malware detection and classification. *Upc.edu*. [online] doi:http://hdl.handle.net/2117/394570.

Or-Meir, O., Nissim, N., Elovici, Y. and Rokach, L. (2019). Dynamic Malware Analysis in the Modern Era—A State of the Art Survey. *ACM Computing Surveys*, [online] 52(5), pp.1–48. doi:https://doi.org/10.1145/3329786.

Rashid, M.U., Qureshi, S., Abid, A., Saad Said Alqahtany, Alqazzaz, A., Hassan, M. ul, Saleh, M. and Shaikh, A. (2025). Hybrid Android Malware Detection and Classification Using Deep Neural Networks. *International Journal of Computational Intelligence Systems*, 18(1). doi:https://doi.org/10.1007/s44196-025-00783-x.

Shaukat, K., Luo, S. and Varadharajan, V. (2023). A novel deep learning-based approach for malware detection. *Engineering Applications of Artificial Intelligence*, 122, p.106030. doi:https://doi.org/10.1016/j.engappai.2023.106030.

Shhadat, I., Bataineh, B., Hayajneh, A. and Al-Sharif, Z.A. (2020). The Use of Machine Learning Techniques to Advance the Detection and Classification of Unknown Malware. *Procedia Computer Science*, 170, pp.917–922. doi:https://doi.org/10.1016/j.procs.2020.03.110.

Venkatraman, S., Alazab, M. and Vinayakumar, R. (2019). A hybrid deep learning image-based analysis for effective malware detection. *Journal of Information Security and Applications*, 47, pp.377–389. doi:https://doi.org/10.1016/j.jisa.2019.06.006.

Vinayakumar, R., Alazab, M., Soman, K.P., Poornachandran, P. and Venkatraman, S. (2019). Robust Intelligent Malware Detection Using Deep Learning. *IEEE Access*, [online] 7, pp.46717–46738. doi:https://doi.org/10.1109/ACCESS.2019.2906934.

Waheed, M. and Qadir, S. (2024). Effective and Efficient Android Malware Detection and Category Classification Using the Enhanced KronoDroid Dataset. *Security and communication networks*, [online] 2024, pp.1–13. doi:https://doi.org/10.1155/2024/7382302.

Wu, C., Shi, J., Yang, Y. and Li, W. (2018). Enhancing Machine Learning Based Malware Detection Model by Reinforcement Learning. *Proceedings of the 8th International Conference on Communication and Network Security - ICCNS 2018*. doi:https://doi.org/10.1145/3290480.3290494.